

Course Outline

Robust and Data-driven Simulation

Probability Theory

Introduction to Simulation

Input probability distributions

Random number generation

Monte Carlo Estimates and Accuracy

Trajectories of Markov Chains

Discrete Event Simulation

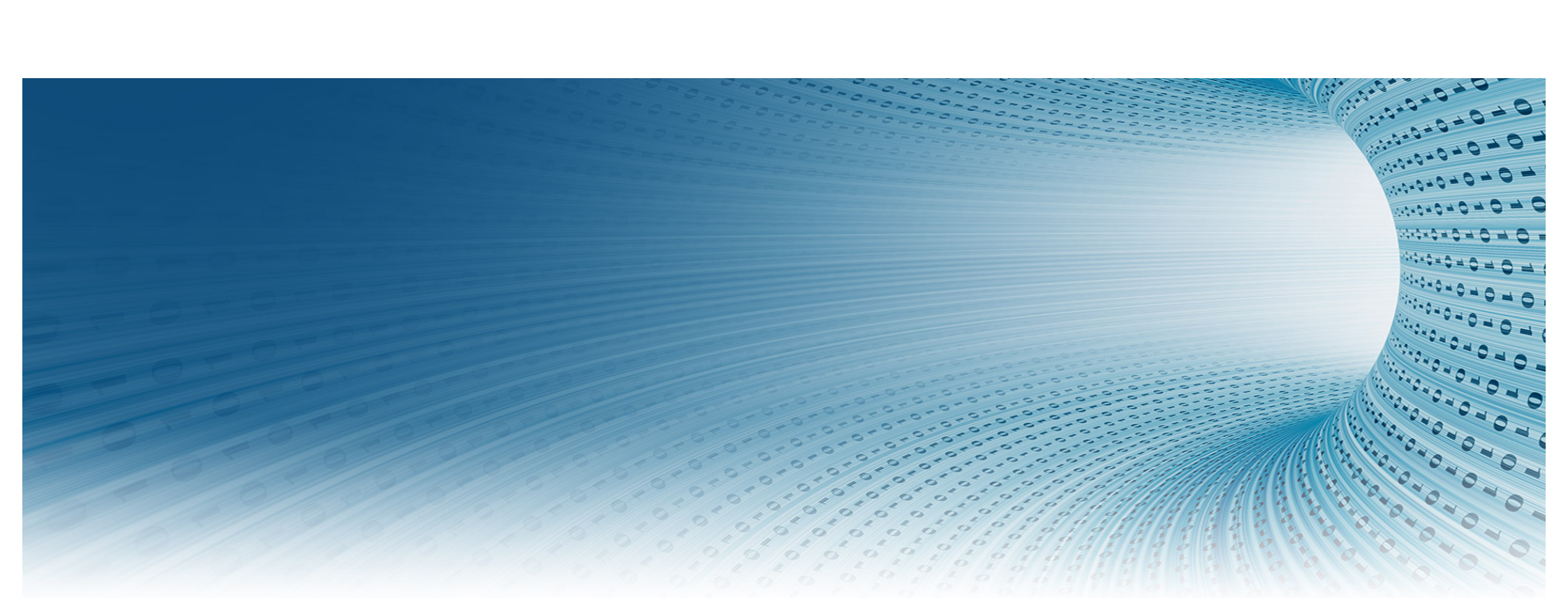
Experimental Design and Optimisation

Robust and Data-driven Optimisation

Chance-constrained optimisation

Stochastic optimisation

Robust optimisation



Discrete Event Simulation

Theory and formulation

Broos Maenhout
Faculty of Economics and Business Administration
Tweekerkenstraat 2
9000 Gent

Outline

- Introduction
- Structure of a Simulation Model
- Examples
 - M/M/I queue
 - A Closed Network of Queues
 - The Machine Repairman problem

Introduction

- Implementation of a discrete-event simulation

- The trajectory of the state of a system Y_n can be generated in a recursive manner (cf. Markov Chains)

$$Y_{n+1} = \phi(Y_n, X_n)$$

with X_n a sequence of independent and identically distributed random variables

ϕ the fixed transition function

- For complex models it is not straightforward to rely on this recursion (too complex)
- Use of simulation models to mimic the dynamics of a system
 - Changes in the state of the system take place on the occurrence of well-defined events which are *discrete* and which take place at discrete instants of time
 - Simulations that use sequences of random number and which are independent of time are called *Monte Carlo simulations*

Introduction

- Generating trajectories

- Time-driven models

- Description of the system dynamics at pre-defined discrete points in time
- Choose (small) time step Δt to generate a sequence of equidistant time epochs
- Y_n is state at time $t_n = n \times \Delta t$
- At each time instant: Update system variables $Y_{n+1} = \phi(Y_n, X_n)$
- Size Δt
 - If the time step is sufficiently small, we can easily relate the system state at consecutive times.
 - The smaller one chooses Δt , the more accurate the calculated trajectory. However, this increases the number of simulation steps and has a negative impact on the CPU time to generate the simulated trajectory.
 - If the time step is too large, multiple events may occur during one time step and a suitable recursion function is required to compute the resulting state of the system.

Introduction

- Generating trajectories

- Event-driven models or next-event time-advance approach

- System only changes state due to certain events (e.g. new customer arrival, product finished) that occur at time points t_k ($k = 1, 2, 3, \dots$)
- Time jumps from event to event: $t_k = t_{k-1} + \tau_k$ (interevent time τ_k)
- Tracking the state at event epochs embodies still a Markov Chain
 - Y_n is the state at 'point n' (e.g. time point n, n^{th} arrival time)
 - Time points are not necessarily equidistant
 - At what points in time should we describe state changes?
 - Discrete events system: Introduce a generic agenda of future events that keeps track of (part of) the state

Introduction

- Generating trajectories

- Event-driven models

- Agenda keeps track of future events

Time (min)	Event	Parameter
0.000	Arrival	...
10.379	Arrival	...
11.298	Departure	...
...		

- Discrete event simulation processes the events in the agenda in chronological order
 - Remove next event from the agenda
 - Execute the corresponding event handler
 - Repeat these steps until the agenda is empty or the time exceeds some bound

Introduction

- Generating trajectories

- Event-driven models

- The state descriptor can be quite complicated and is deducted from the state information and the state of the agenda
 - In the extreme case for a queueing network, we have the following inputs
 - virtual customers with properties
 - residing in buffers (say, in lists)
 - customers move from buffer to buffer on discrete events

Introduction

- Remark: Markovian property is difficult to obtain for complex systems
 - Future behaviour is independent from the past
 - Choice of distribution: Not always very realistic
 - Queueing systems are typically investigated by making use of exponential distributions (service times) or Poisson distributions (arrival process). As a result of the memoryless property we do not have to track remaining service times or interarrival times.
 - However, for non-exponential random variables the distribution is not memoryless: The rate at which an event occurs varies with the length of time since the previous occurrence of the event. Interevent times are not memoryless and should therefore be generated independently (as soon as possible after the preceding occurrence). As a result several interevent times will be active.
 - Independency: Correlation between events
 - Service times of related jobs on different machines may be correlated
 - Service times are often not independent because of machine deterioration

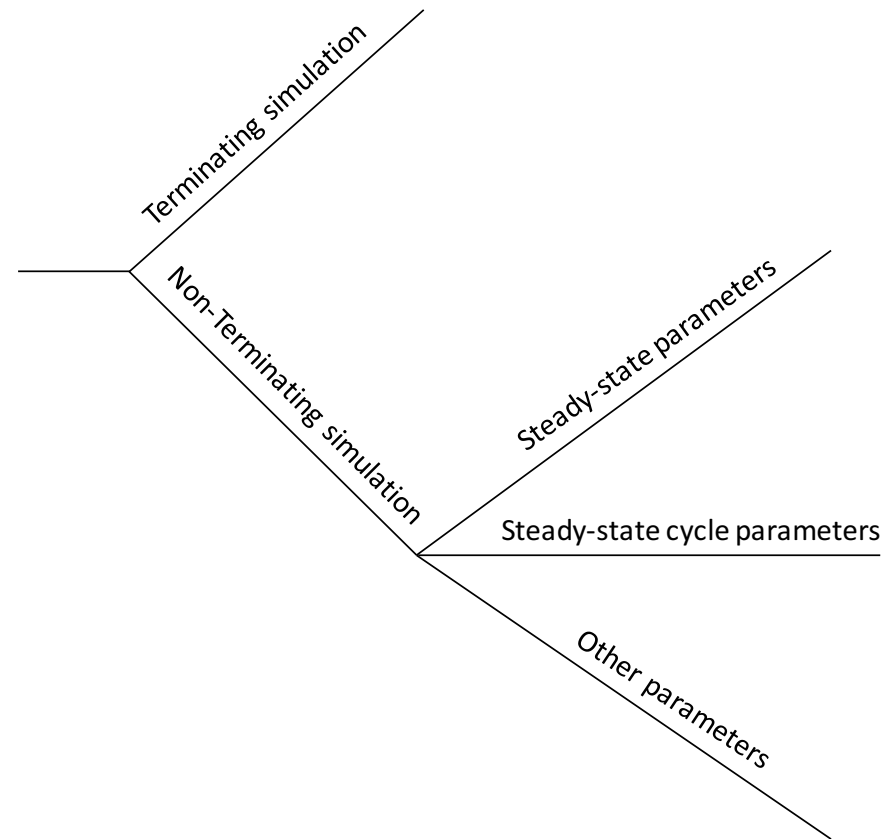
Introduction

- Types of Simulations with regard to output analysis
 - Across versus within-replication data from across-replication data
 - Within-replication data
 - Y_{ij} jth observation in the ith replication
 - Across-replication data are formed by summarising within-replication data
 - $\bar{Y}_i$ average of the ith replication
 - Y_i^{max} maximum value in the ith replication over all observations
 - Across-replication data are independent (different random numbers) and identically distributed (same model),
 - In Markov Chains, within-replication data are not independent and identically distributed
(E.g. cycle time for parts and work in process (WIP), waiting time)

=> Multiple Markov trajectories need to be evaluated

Introduction

- Types of Simulations with regard to output analysis



Introduction

- Types of Simulations with regard to output analysis

- Terminating simulation

- A simulation for which there is a natural event that specifies the length of each run (E.g. the time point when the system is cleaned out, at a time point beyond which no useful information is obtained or a time point specified by the management)
 - The number of observations can be constant or random
 - The distribution between different states may change over time (e.g. the distribution of the arriving customers)
 - Example: A bank opens at 8:30 am (time 0) with no customers present and 8 of the 11 teller working (initial conditions) and closes at 4:30 pm (terminating condition)
 - When to use?
 - Simulation of non-ergodic Markov Chains, i.e. Markov Chains that do not reach steady-state
 - Simulation of output parameters not related to steady-state
 - Simulation of systems with independent observations within a single simulation run

Introduction

- Types of Simulations with regard to output analysis
 - Terminating simulation: Method
 - Conduct n independent replications with independent seeds and the same initialisation rule
 - The initial conditions at time 0 affect the measures of performance and are representative for the actual system
 - How should the simulation be initialized? Suppose that a machine shop closes at a certain time each day, even if there are jobs waiting to be served. You have to be careful to start each day with a demand that depends on the number of jobs remaining from the previous day.

Introduction

- Types of Simulations with regard to output analysis

- Non-terminating simulation

- A simulation for which there is no natural event to specify the length of the run, i.e. the system runs continuously
- Example: assembly lines that shut down infrequently, telephone systems
- This simulation may enable us to analyse the following parameters, i.e.
 - Steady-state parameters (e.g. the yearly throughput with a fixed arrival pattern, waiting time in steady-state) that are not influenced by the initial conditions of the model
 - Steady-state cycle parameters

If you do not have steady state, you might have a steady cycle (a cycle repeating over time)

Times axis can be divided into time interval cycles (e.g. one week in a call center, one week in an emergency department)

Y_i^C random variable on i-th cycle (e.g. number of calls with a waiting time longer than 15 minutes in week i in a call center)

$Y_1^C, Y_2^C, Y_3^C \dots$ has a steady state distribution

(e.g. the average hourly throughput when the arrival pattern changes from hour to hour but is identical for different weeks)

- Other parameters (e.g. throughput if arrival rates change from hour to hour, week to week)

Introduction

- Types of Simulations with regard to output analysis
 - Non-terminating simulation
 - Consider a single run of a simulation model to estimate a steady-state or long-run characteristics of the system: The single run produces observations $Y_1, Y_2, \dots$
 - Performance measures
 - Point estimator and confidence interval
 - Independent of initial conditions
 - The sample size is a design choice, with several considerations in mind, i.e.
 - Any bias in the point estimator that is due to artificial or arbitrary initial conditions (bias can be severe if run length is too short)
 - Desired precision of the point estimator
 - Budget constraints on computer resources

Introduction

- Types of Simulations with regard to output analysis

- Non-terminating simulation

- Initialisation bias

- No widely accepted, objective and proven technique to guide how much data to delete to reduce initialisation bias to a negligible level
- Plots can be misleading but are still recommended
 - If averages reveal a smoother and more precise trend as the # of replications increases.
 - If averages can be smoothed further by plotting a moving average.
 - Cumulative average becomes less variable as more data are averaged.
 - The more correlation present, the longer it takes for to approach steady state.

- Replication method

- Batch Means method
- Regenerative method

Structure of a Simulation Model

- The State Descriptor
- Events and their management
- Simulation Time
- Program Structure

Structure of a Simulation Model

- The State Descriptor
 - The representation of the system at a particular time is called the state descriptor vector or the system state
 - The system state is a function of time
 - The system can be represented by a single element or a vector consisting of different components/variables
 - Each component can be viewed as a discrete random variable
 - Example: The number of customers waiting in a M/M/I queue

Structure of a Simulation Model

- Events and their management
 - Define all possible events and their effect on the system state
 - The occurrence of an event can cause one or multiple components of the system state to change values
 - Example: Consider an M/M/1 queue there are only two events
 - System state: A single integer to represent the number of customers present
 - Events:
 - A new customer arrives at the queue: Increment the system state by 1
 - A customer completes service and departs: Decrement the system state by 1
 - In a single server feedback queue in which customers re-enter the queueing system with probability α , the service completion will leave the system change unchanged with probability α
 - As events succeed each other in time, an *event list* is kept which holds the time for the next occurrence of each different event

Structure of a Simulation Model

- Simulation Time

- Wall-clock time (run time)

- Definition: CPU time required by a computer to execute the simulation program (platform-specific)

- Internal clock time

- Definition: Duration T of the simulation experiment or the time over which the model is exercised
 - Stop criterion: Fixed period of time ($= T$) (clock time) or the time until a pre-specified number of one or more events occurred (clock count)
 - The variable t indicates the passage of time within the simulation model/real system
 - Advancement of the internal clock time
 - Synchronous simulation: The internal clock is updated at fixed increments of time (Disadvantage: Length time increment?)
 - Asynchronous simulation: The internal clock is update at the occurrence of events

Structure of a Simulation Model

- Simulation run
 - Definition: Each execution of a simulation from the point at which it is initiated ($t = 0$) until it is halted ($t = T$) and defines a sample path
 - A sample path is the evolution over time $t \in [0, T]$ of the state descriptor vector
 - Event explosion occurs if the simulation is not stable in the sense that per unit of simulation time, more events are created than handled (this means that your system is not ergodic)

Structure of a Simulation Model

- Program structure

- Initialisation routine

- Initialise the simulation model at time 0

- Generate a method for sequencing event occurrences at appropriate times (main routine)

- Initialise variable for a new simulation run
 - Select the next event to occur
 - Call the event module if required
 - Determine when the current simulation finishes

- Generate a method for each type of event (event handler module)

- Update internal clock
 - Change the system state (event routine)
 - Generate the time of the next occurrence of this event (timing routine)

Structure of a Simulation Model

- Program structure

- Generate a method for each type of event (event handler module)

- The event handlers should collect the necessary statistics
- Let τ_n be the time of the n^{th} event and let $Y(t)$ be the state of the system at time t
- Calculate the sample sum

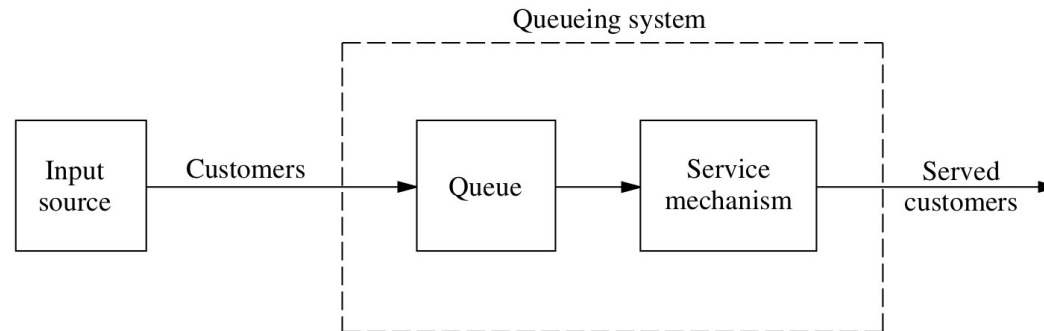
$$E[f(Y(\tau))] = \frac{1}{N} \sum_{n=1}^N f(Y(\tau_n))$$

- When do we observe the state?
 - Observing the state at arrival times or at departure times may yield different results
 - Neither arrival or departure times are 'random'
 - You may generate random observation times via a Poisson distribution with an average mean (high rate: many correlated observations – low rate: too few observations)
- How to set the stopping criterion?
 - Simulation should be long enough to generate useful and representative samples
 - Depends on the performance measure

Example: M/M/1 queue

- Single-server queue with first-come first-served scheduling of customers

➤ Representation



- Analytical formulas for each model indicate how the corresponding queueing system *should* perform: $W_{n+1} = \max(W_n + S_n - A_n, 0)$
- We use a simulation model to study the waiting performance of a system

Example: M/M/I queue

- M/M/I queue

- Input source models the customer arrival process

- Distribution of Arrivals: The random arrival of customers into the system is modeled as a Poisson process:

$$P(X = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$

where

λ = mean arrival rate per time unit

t = the length of the interval

i.e. interarrival times have an exponential distribution with mean $1/\lambda$

- Queue: The queue is where customers wait before being served (infinite queue)

- Queue discipline: First come, first serve

Example: M/M/I queue

- M/M/I queue

- Service mechanism

- 1 server
- Service time
 - The time required for the service from the commencement to the completion
 - The service time are distributed according to the exponential distribution

$$f(t) = \mu e^{-\mu t}$$

where

μ = the average number of customers who can be served per time period ($1/\mu$ = the mean service time)

t = the length of the interval

- The probability that the service time is less than some t :

$$P(X \leq t) = 1 - e^{-\mu t}$$

Example: M/M/I queue

- Structure of the simulation model
 - The State Descriptor
 - The number of customers in the system (indicated by a single integer n)
 - Events
 - The arrival of a new customer
 - If the queue is empty, schedule the departure at a sampled service time from now
 - Increase the queue size by one
 - Schedule the next arrival at a sampled interarrival time from now
 - Departure of a served customer
 - Decrease the queue size by one
 - If the queue is not empty, schedule the next departure at a sampled service time from now

Example: M/M/I queue

- Program structure

- Simulation Time

- Internal clock time is indicated by t .
 - The variables t_a and t_d denote the scheduled times of the next arrival and the next departure.
 - The variables t_λ and t_μ denote the generated interarrival times and service times.
 - Stop criterion: a maximum number of served customers (N)

Example: M/M/1 queue

- Program structure

- Evaluated output

- Discrete-time statistic vs Continuous-time statistic

- Discrete-time statistic: Statistics collected based upon a number of random variables that have a discrete time index ($i = 1, 2, \dots$)
(e.g. mean waiting time, maximum waiting time)
- Continuous-time statistic: Statistics collected based upon a number of random variables indexed on the continuous time parameter t
(e.g. number in queue, server utilisation, proportion of time more than 5 customers in the queue)

- We calculate the mean interarrival time, the mean service time and the mean time spent waiting in the queue

- The variables tot_λ and tot_μ denote the running totals of the arrival times and the service times
- Time spent in the system of customer $i = (\text{Departure time}(i) - \text{Arrival time}(i))$
 $\Rightarrow \widehat{\text{System time}} = \sum (\text{Departure time}(i) - \text{Arrival time}(i)) / n$ (Discrete-time statistic)
- Average queueing time = Average time in the system – Average service time

Example: M/M/1 queue

- Structure of the simulation model

- Pseudocode Event-driven simulation

Initialise parameters ()

```
t = 0; N = 100;  $\lambda$  = 3;  $\mu$  = 5; // Initialise relevant input values and counters
nd = na = n = 0; time_system = 0;
td = infinity; t $\lambda$  = t $\mu$  = tot $\lambda$  = tot $\mu$  = 0;
r = rand(); ta = -ln(r)/ $\lambda$ ; tot $\lambda$  = ta; // Initialise time of first arrival
for k in range (1,N+1) { // Initialise array of arrival times
    time_arrival[k] = 0;}
}
```

Simulation procedure()

```
while (nd < N){
    if (ta < td) { // Arrival event
        t = ta; n++; na++; if (na <= N) {time_arrival[na] = t;}
        r = rand(); t $\lambda$  = -ln(r)/ $\lambda$ ; ta = t + t $\lambda$ ; tot $\lambda$  += t $\lambda$ ;
        if (n == 1) { // Arrival to an empty system
            r = rand(); t $\mu$  = -ln(r)/ $\mu$ ; td = t + t $\mu$ ; tot $\mu$  += t $\mu$ ;}
        else { // Departure event
            t = td; n--; nd++;
            time_system += t - time_arrival[nd];
            if (n == 0) {td = infinity;}
            else {
                r = rand(); t $\mu$  = -ln(r)/ $\mu$ ; td = t + t $\mu$ ; tot $\mu$  += t $\mu$ ;}
            }
        }
}
```

Example: M/M/1 queue

- Structure of the simulation model

- Pseudocode Event-driven simulation

```

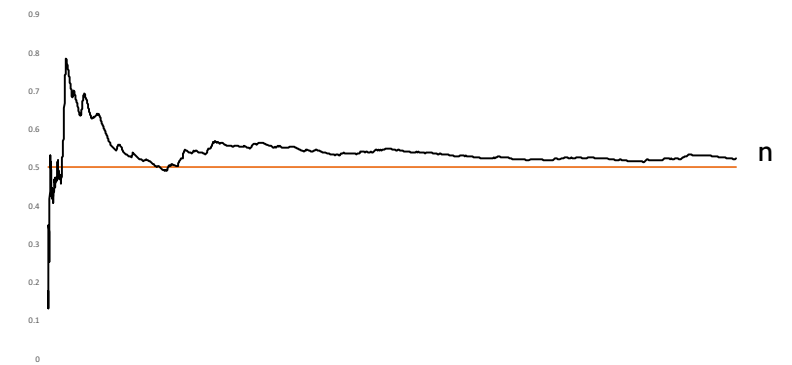
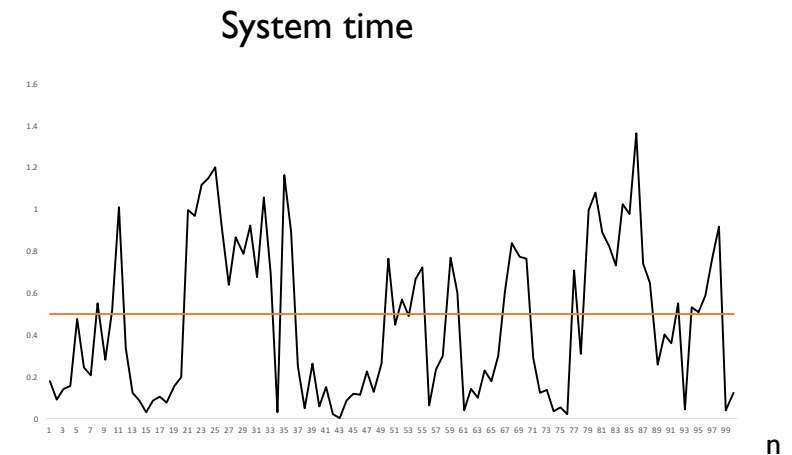
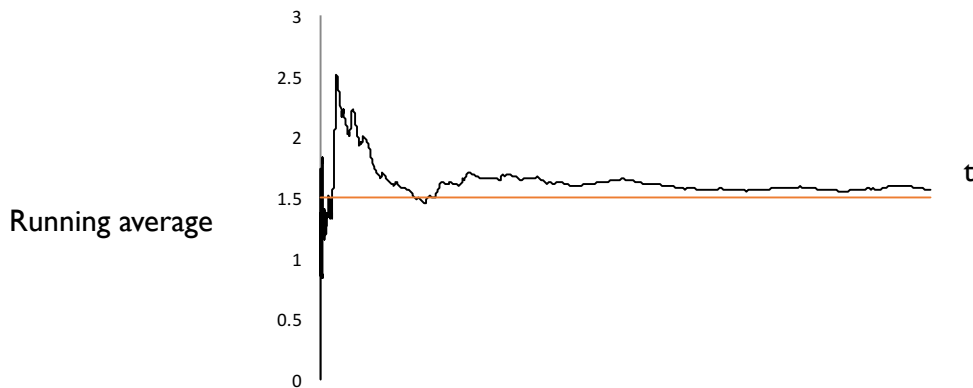
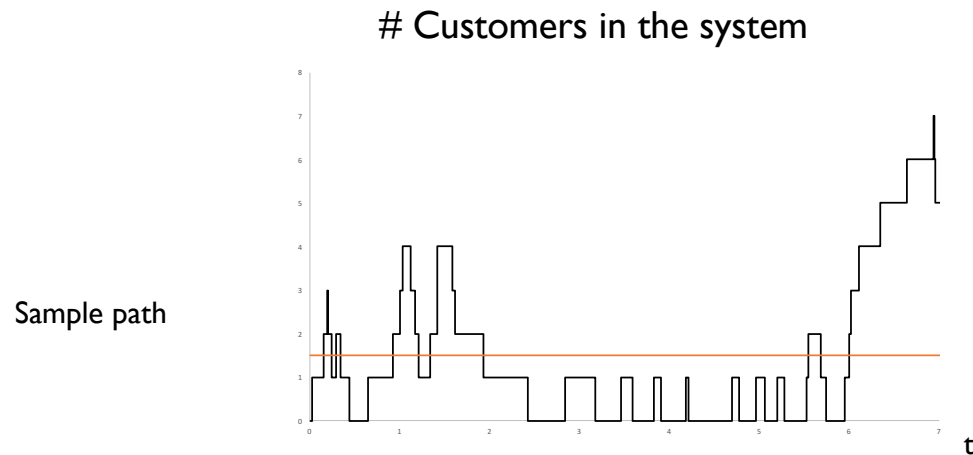
Output(){
    mean_waiting_time = (time_system - totμ)/N;           // Calculate average waiting time
    mean_interarrival_time = totλ /N;                     // Calculate average interarrival time
    mean_service_time = totμ/N;                           // Calculate average service time
    mean_system_time = mean_waiting_time + mean_service_time
}
    
```

- Output: $\lambda = 3$; $\mu = 5$

N	Interarrival time	Waiting time	Service time	System time	# Customers in system	# Customers in queue
100	0.344	0.254	0.210	0.463	1.361	0.745
1000	0.329	0.351	0.199	0.550	1.682	1.077
10000	0.333	0.322	0.200	0.522	1.567	0.967
Theoretical	0.333	0.3	0.2	0.5	1.5	0.9
	$= 1/\lambda$	$= \lambda/(\mu(\mu-\lambda))$	$= 1/\mu$	$= 1/(\mu-\lambda)$	$= \lambda/(\mu-\lambda)$	$= \lambda \times \lambda/(\mu(\mu-\lambda))$

Example: M/M/1 queue

- Structure of the simulation model
 - Output: Sample paths and running averages



Example: M/M/I queue

- Some extensions/modifications
 - Non-exponential interarrival and service time distributions
 - Use another generation method (inversion method or acceptance/rejection method) to generate a random observation from a probability distribution
 - Limited waiting room (M/M/I/K)
 - The arrival event needs to be modified since some customers will not be able to enter the queue and a counter is required to count the lost number of customers.
 - Multiple servers (M/M/c)
 - If upon arrival the number of customers n is less than c , the arriving customer can go immediately into service
 - In the departure event, we should keep track of the time at which customer i enters service (`time_service[i]`)

Example: M/M/1 queue

- Method for accurate estimation: Batch mean method

- Example: Waiting times in a queue

- When using a single, long replication, there may arise the following problem:
 - Data are dependent of the initial state, which leads to a biased estimator
 - Replication method, i.e. more than 25 replication runs) is too expensive => Apply Batch Means method (if necessary delete warm-up period) or Regenerative method
 - Especially relevant for output measures that concern a single run, e.g., the time needed to process 1000 tasks, the maximal queue length
- Input
 - Arrival process: Poisson distributed with interarrival rate $\lambda = 0.09$
 - Service process: Exponentially distributed with service rate $\mu = 0.1$

=> Limiting mean customer delay $E[D] = v = \frac{\lambda}{\mu(\mu-\lambda)} = 90$

Example: M/M/I queue

- Method for accurate estimation: Batch mean method

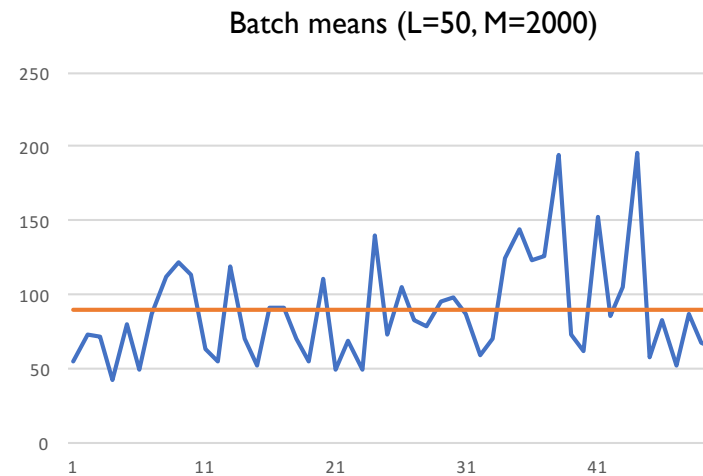
- Example: Waiting times in a queue

- Fixed batch size

- Number of batches $L = 50$
- Batch size $M = 2000$ customers
- Batch means $\bar{D}_l = \frac{1}{M} \sum_{k=(l-1)M+1}^{lM} D_k$ (cf. Figure)
- Overall mean $\bar{\bar{D}} = \frac{1}{L} \sum_{l=1}^L \bar{D}_l = 88.64$
- $\text{Var}[\bar{\bar{D}}] = \frac{1}{L-1} \sum_{l=1}^L (\bar{D}_l - \bar{\bar{D}})^2 = 35.15$
- 95% confidence interval: [78.89; 98.38]
- Coverage probability = 0.1

- Other batch sizing strategies in Alexopoulos and Seila (1996)

- E.g. SQRT-rule: Batch size = $\lfloor \sqrt{N} \rfloor$; Number of batches = $\lfloor \sqrt{N} \rfloor$ (e.g. $N = 100000$: $M = 316$ and $L = 316$)



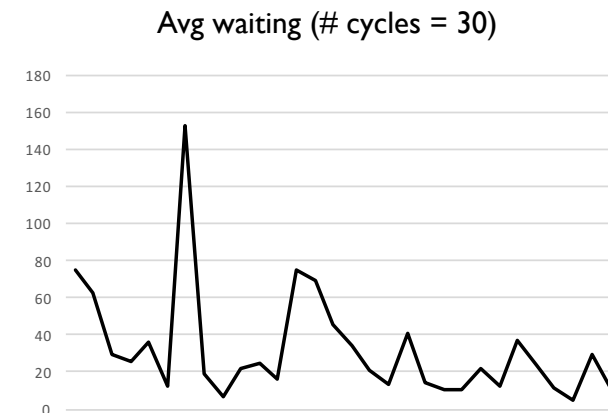
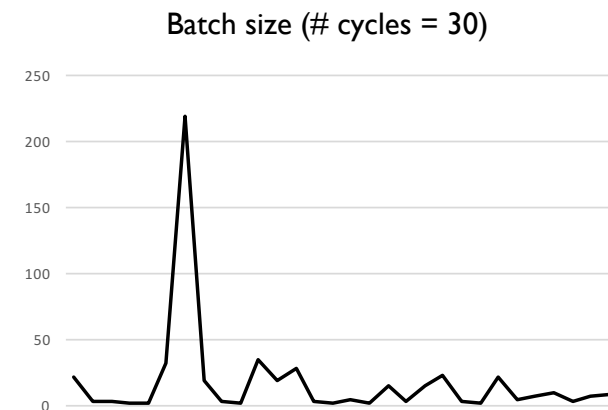
Example: M/M/I queue

- Method for accurate estimation: Regenerative method

- Example: Waiting times in a queue

- Variable number cycles
 - Number of customers $N = 100000$
 - Regeneration state = 5 customers in system leading to 6558 cycles
 - Example: Sample observations first 10 cycles

Batch	Batch_size	λ	μ	Wq	L_q
1	21	11.49	10.38	74.98	7.02
2	2	8.24	6.07	62.61	4.43
3	2	11.94	9.55	29.56	3.94
4	1	6.93	13.51	25.42	3.57
5	1	14.59	12.20	36.30	3.72
6	31	15.30	9.85	11.89	0.54
7	218	10.32	10.19	152.45	14.84
8	18	16.31	10.36	19.31	0.64
9	2	6.55	3.13	7.07	4.11
10	1	6.03	22.15	21.33	3.66



Example: M/M/1 queue

- Method for accurate estimation: Regenerative method

- Example: Waiting times in a queue

- Variable number cycles

- $\bar{Y} = \frac{1}{\tilde{N}} \sum_{n=1}^{\tilde{N}} \tilde{Y}_n^{(\ell)} = 1343.79$ (average total waiting time per batch)
 - $\bar{R} = \frac{1}{\tilde{N}} \sum_{n=1}^{\tilde{N}} R_n^{(\ell)} = 15.25$ (average number of customers per batch)
 - $\bar{X} = \frac{\sum_{n=1}^{\tilde{N}} \tilde{Y}_n^{(\ell)}}{\sum_{n=1}^{\tilde{N}} R_n^{(\ell)}} = 88.12$
 - $\text{var}[\bar{X}] = 105720909.76$
 - 95% confidence interval: [83.95; 92.31]

More accurate than Batch Means method

Example: A closed network of queues

- A closed network of single-server queueing systems with first-come first-served scheduling of customers

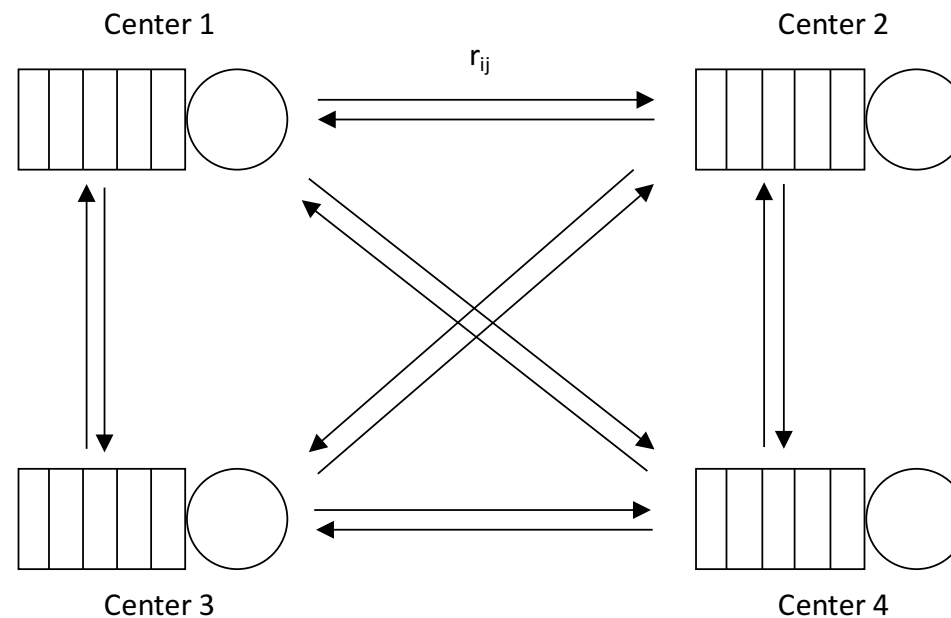
➤ Problem description

- Consider a network of M service centers
- Each center contains a single queue and feeds a single serve
- Service times are exponentially distributed but may be different at different service centers
- A fixed number of customers N circulates among these centers: after leaving service center i , a customer proceeds to service center j with probability r_{ij} .
- Customers do not arrive from the exterior nor do they leave the network
- Objective: Identify the bottleneck center, i.e. the center with the most customers

Example: A closed network of queues

- A closed network of single-server queueing systems with first-come first-served scheduling of customers

➤ Representation



Example: A closed network of queues

- Structure of the simulation model

- The State Descriptor

- A vector with a length equal to the number of service centers. The i^{th} component indicates the number of customers in the service center i (indicated by a single integer n)

- Events

- The arrival of a new customer
 - A customer arrives at another center j with probability r_{ij} after the customer departs in service center i ,
i.e. departure time of customer k in center i = arrival time of customer k in center j
 - If the queue is empty, schedule the departure at a sampled service time from now
 - Increase the queue size by one
 - Departure of a served customer
 - Decrease the queue size by one
 - If the queue is not empty, schedule the next departure at a sampled service time from now

Example: A closed network of queues

- Program structure

- Simulation Time

- Internal clock time is indicated by t .
 - The array $t_d[i]$ denotes the scheduled times of the the next departure at service center i
 - The variables $t_\mu[i]$ denotes the generated service times at service center i
 - Stop criterion: a maximum duration (T)

- Evaluated output

- We calculate the mean interarrival time, the mean service time and the mean time spent waiting in the queue for each service center i

Example: A closed network of queues

- Structure of the simulation model

- Pseudocode Event-driven simulation

```
Initialise parameters (){
  all variables are set equal to 0;
  t = 0;
  Initialise M, N, T,  $\mu$ [ ], r[ ][ ]
  j = 0;
  for i in range (0, M-1){
    n[i] = rand()%(N-j)
    j += n[i]}
  n[M-1] = N-j;
  for i in range (0, M){
    td[i] = infinity;
    if (n[i] > 0)
      r = rand(); td[i] = -ln(r)/ $\mu$ [i];
  }
```

// Initialise relevant input values and counters

// Initialise number of customers in each service center

// Initialise time of first departure

Example: A closed network of queues

- Structure of the simulation model

- Pseudocode Event-driven simulation

```
Simulation procedure(){
  while (t < T){
    first_depart = infinity;
    for i in range (0, M){
      if (td [i] < first_depart){
        first_depart = td [i]; source = i;}}
    r = rand(); j = 0; dest = 0;
    while (j < r){
      j += r{source}{dest}; dest++;}
    t = td[source];
    n[source]--; n[dest]++;
    if (n[source] > 0)
      r = rand(); td[source] = -ln(r)/μ[source];
    if (n[dest] > 0)
      r = rand(); td[dest] = -ln(r)/μ[dest];

Output(){
  // Calculate average waiting time for each center
  // Calculate average interarrival time for each center
  // Calculate average service time for each center
  // Calculate number of customers in each center
}
```

Example: A closed network of queues

- Experimentation

- Input

- Number of customers $N = 10$
 - Number of service centers = 4
 - Service rate: $\mu_1 = 3, \mu_2 = 1, \mu_3 = 0.75, \mu_4 = 2$
 - Transition probabilities r_{ij}

r_{ij}	0	1	2	3
0	0.2	0.2	0	0.6
1	0.5	0.5	0	0
2	0	0.4	0	0.6
3	0	0	1	0

- $T = 100$

Example: A closed network of queues

- Computational results

- Output: Overall results

T	System	Interarrival time	Waiting time	Service time	System time	# Customers	# Customers
100	Q1	2.280	0.094	0.461	0.555	0.233	0.039
	Q2	1.605	0.983	1.086	2.069	1.411	0.725
	Q3	1.172	7.780	1.156	8.937	7.929	6.921
	Q4	1.143	0.151	0.431	0.581	0.506	0.131
10000	Q1	2.063	0.065	0.341	0.407	0.197	0.032
	Q2	1.287	2.569	0.988	3.557	2.763	1.996
	Q3	1.389	7.640	1.344	8.984	6.471	5.503
	Q4	1.388	0.286	0.504	0.790	0.569	0.206

⇒ Service center 3 is the bottleneck system (highest system time)

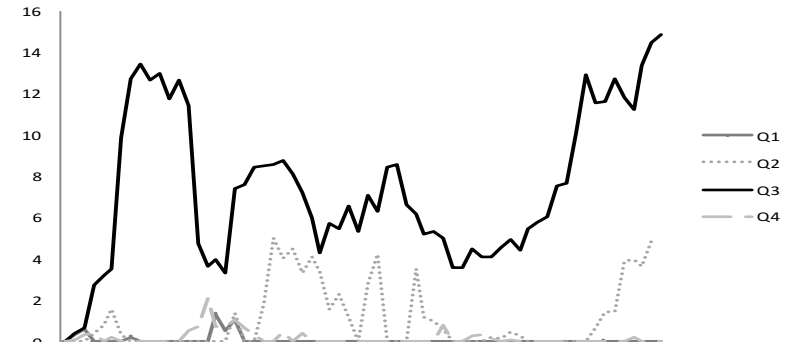
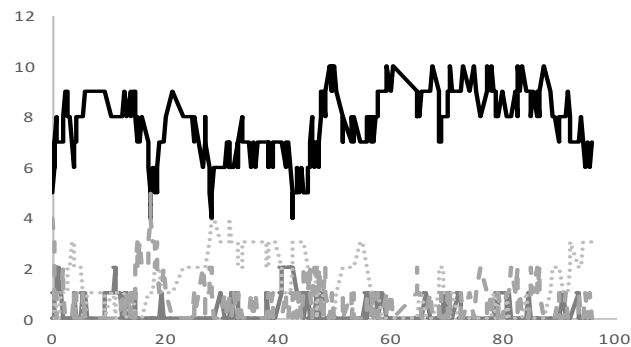
Example: A closed network of queues

- Computational results
 - Output: Sample paths and running averages

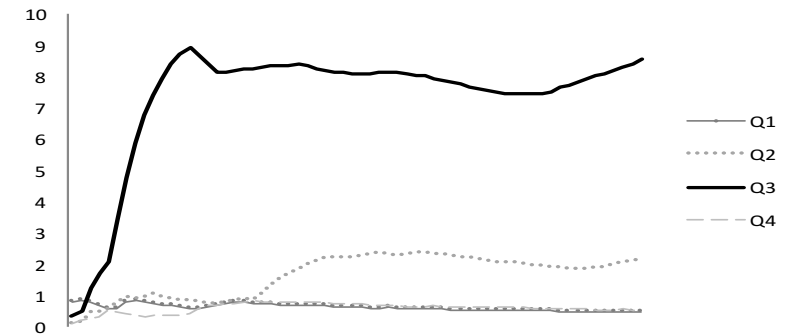
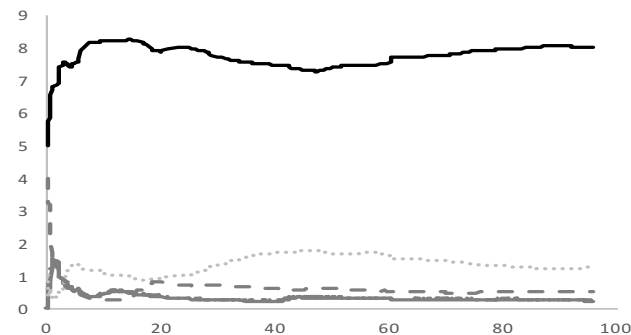
Customers in the system

System time

Sample path



Running average



Example: The Machine Repairman Problem

- Problem description
 - A factory possesses a total of M identical machines that are subject to breakdown
 - A certain minimum number of working machines $N < M$ is required for the factory to function correctly
 - When less than N of the machines are in working order, the factory is forced to stop production
 - Machines in working order in excess of the N needed for the factory to keep functioning are kept as spares
 - Machine breakdown
 - When a machine breaks down, it is immediately replaced by a spare (if there is one available) and the broken machine is set aside for repair
 - Machine break down independently
 - $B(t)$ is the general probability distribution function of the time from the moment a machine is placed in service until it breaks down

Example: The Machine Repairman Problem

- Problem description

- Machine repair

- A single repair man is available and he repairs broken machines one at a time
 - The distribution of repair time has a general probability distribution function $R(t)$
 - Once repaired, a machine immediately becomes available as a spare

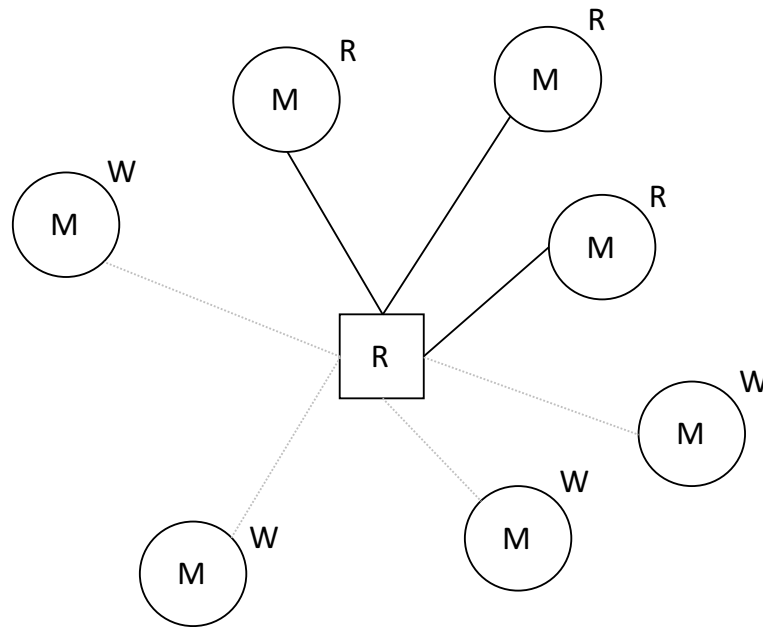
- Spare machines are not used and so do not break down

- Objective: Calculate the length of time until the number of working machines first falls below N and the factory is forced to halt the production

Example: The Machine Repairman Problem

- Program structure

- Representation



Example: The Machine Repairman Problem

- Structure of the simulation model
 - The State Descriptor
 - The number of machines that are in working order (denoted by n) or the number of machines broke down (denoted by b). Since $n + b = M$ only one of these two variables is actually needed
 - Events
 - A machine breakdown
 - The distribution of machine breakdown times needs not to be exponentially distributed. Hence, it is required to keep the next breakdown time for each of the N working machines
 - Store the machine breakdown times in a list $t_b[i]$ ($\forall i$)
 - Decrement the number of working machines and increment the number of broken machines
 - If there is a spare machine available ($n > N$), put it into service and generate the time at which this new machine will fail
 - If the repairman was idle before this machine breakdown, generate the time at which the repair will be completed

Example: The Machine Repairman Problem

- Program structure

- Events

- A machine repair
 - The time until the next repair completion is equal to ∞ if $b = 0$
 - Increment the number of working machines and decrement the number of broken machines
 - If there is a another machine to be repaired compute the time at which the repair will be finished.

- Simulation Time

- Internal clock time is indicated by t .
 - Stop criterion: the simulation is stopped when the number of working machines $n < \text{the required number of working machines } N$ and production is halted

- Evaluated output

- We calculate the simulation time (the time before the factory is halted)

Example: The Machine Repairman Problem

- Structure of the simulation model

- Pseudocode Event-driven simulation

```
Initialise parameters (){
    all variables are set equal to 0;
    t = 0;
    Initialise M, N, infinity;
     $\mu_b = 600$ ;  $\sigma_b = 100$ ;
     $\mu_r = 15$ ;  $\sigma_r = 10$ ;
    n = M; b = 0;
    for i in range (0, N){
        r = rand(); x = NORMINV(0, 1, r);
         $t_b[i] = \sigma_b * x + \mu_b$ ;
         $t_r = \text{infinity}$ ;
    }
}
```

// Initialise relevant input values and counters

// Initialise time of first breakdown for each working machine
// We make use of the Polar method of Shelton Ross

// Initialise time to first repair

Example: The Machine Repairman Problem

- Structure of the simulation model

- Pseudocode Event-driven simulation

```
Simulation procedure(){
  while (n >= N){
    first_breakdown = infinity;
    for i in range (0, N){
      if ( $t_b[i] < \text{first\_breakdown}$ ){
        first_breakdown =  $t_b[i]$ ; index= i;}}
    if ( $t_r < \text{first\_breakdown}$ ){
      t =  $t_r$ ; n++; b--;
       $t_r = \text{infinity}$ ;
      if (b > 0) {
        r = rand(); x = NORMINV(0, 1, r);
         $t_r = t + \sigma_r * x + \mu_r$ ; }}
    else {
      t =  $t_b[\text{index}]$ ; n--; b++;
      if (n > N)
        r = rand(); x = NORMINV(0, 1, r);
         $t_b[\text{index}] = t + \sigma_b * x + \mu_b$ ;
      if (b == 1)
        r = rand(); x = NORMINV(0, 1, r);
         $t_r = t + \sigma_r * x + \mu_r$ ; }}}
  }
```

// Determine first breakdown event

// Next event is repair completion

// Next event is a machine breakdown

// Get time until new machine fails

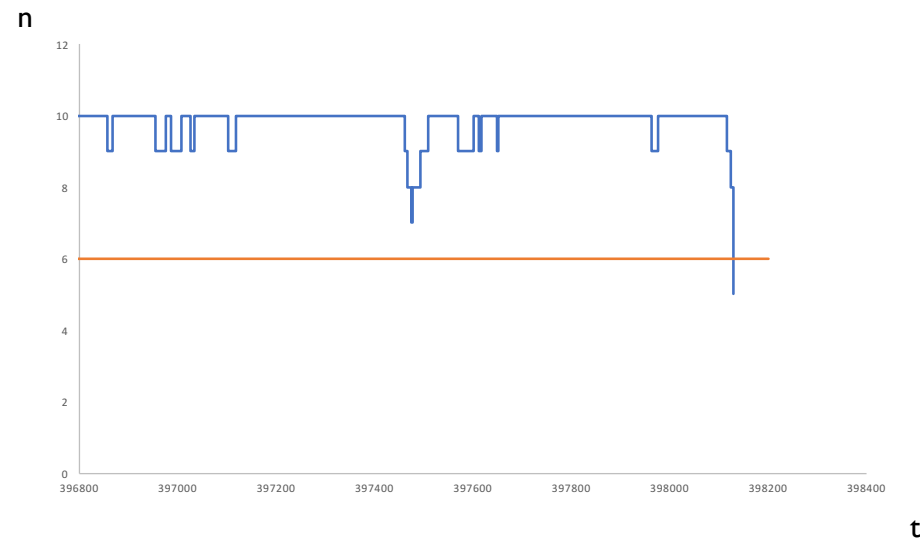
// Repair queue was empty. Generate repair time

Example: The Machine Repairman Problem

- Test design
 - Parameter values
 - $M = 10$
 - $N = 6$
 - Breakdown time $\sim N(600, 100)$
 - Repair time $\sim N(15, 10)$
 - Number of simulation runs: 10000

Example: The Machine Repairman Problem

- Computational results
 - The factory is halted when $n < 6$
 - $\bar{X} = 272825$
 - $S = 276565$
 - 95% confidence interval = [267404; 278246]
 - Sample path (number of working machines)



Example: The Machine Repairman Problem

- Computational results

- Sensitivity analysis: Number of machines – Number of required working machines

- Number of machines (M)

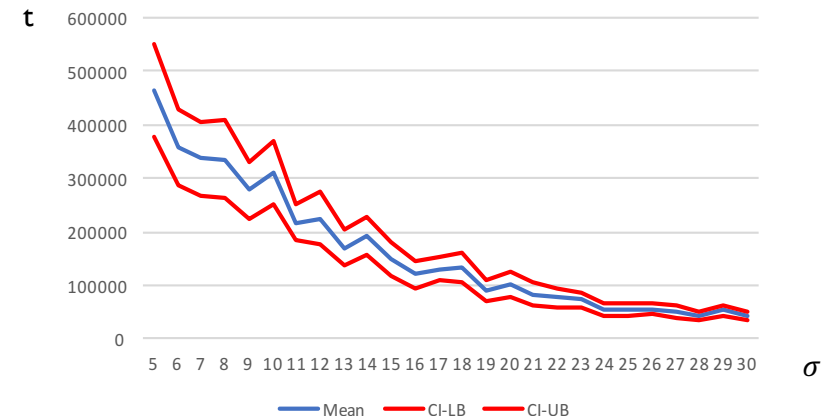
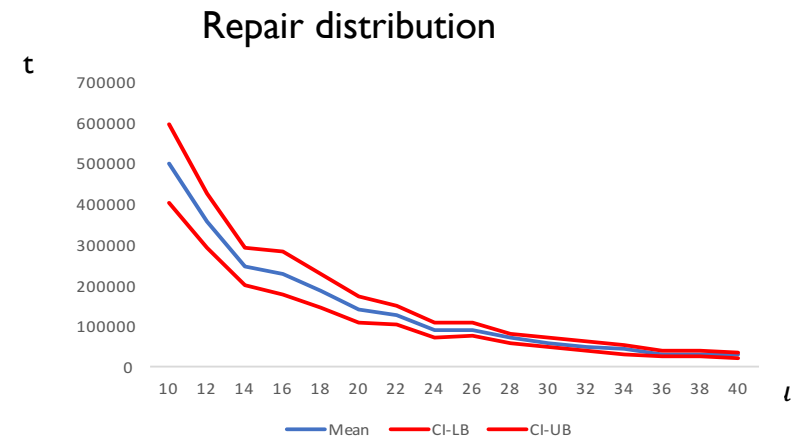
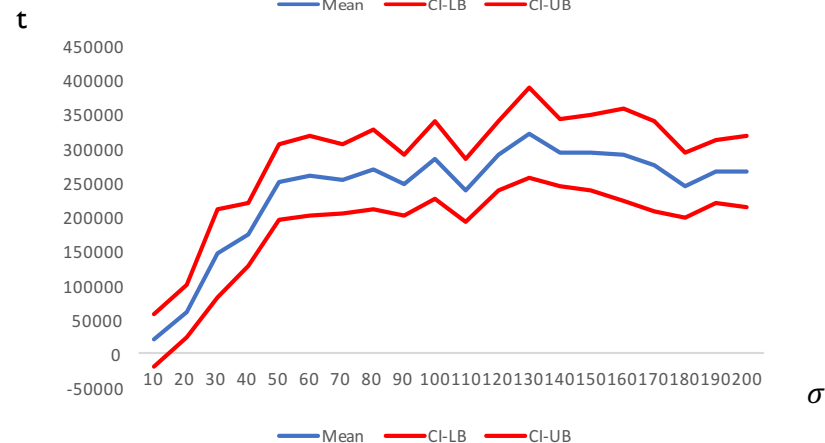
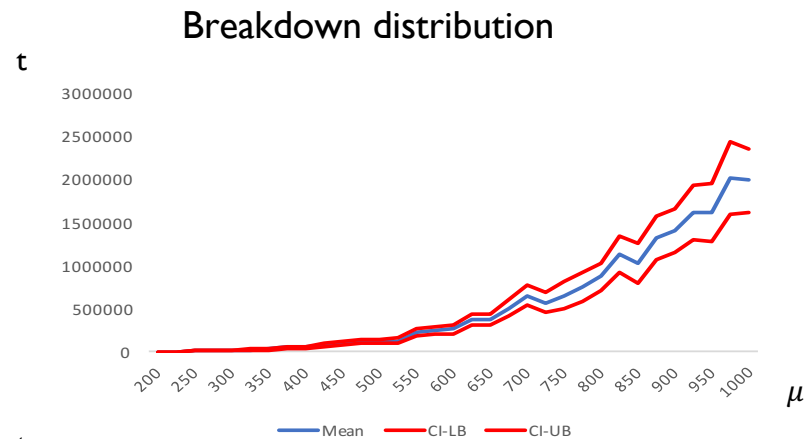
N	Mean	ST DEV	CI-	CI+
6	463	67	449	476
7	478	63	466	491
8	788	406	708	867
9	13366	14035	10615	16117
10	262812	295362	204921	320703
11	11199516	11099776	9023960	13375073

- Number of required working machines (N)

N	Mean	ST DEV	CI-	CI+
5	70486783	80874725	54635337	86338229
6	327016	275688	272981	381051
7	6739	7601	5249	8229
8	611	170	578	645
9	451	58	440	462

Example: The Machine Repairman Problem

- Computational results
 - Sensitivity analysis: Probability distributions



Example: The Machine Repairman Problem

- Possible extensions

- An additional repairmen

- In this case, the state descriptor needs to include the number of busy (or idle) repairmen. A list should be created for the repairmen indicating the finish repair time. If a machine breaks down while at least one repairman is idle, repair on that machine can begin immediately.

- Calculate the percentage of unproductive time

- In this case the simulation continues until some predetermined ending time T and the sum of the lengths of time for which $n < N$ is computed. The ratio of this to T can be used to compute the percentage of down-time.

- Different types of machines

- There are possibly different types of machines and some limited number of each is needed for production. Spare machines of different types are kept on standby.
 - Machines of different types may have different breakdown and repair time distributions and repairman may be specialised to handle just one or two different types of machines
 - The objective is to determine the optimal number of spares of each machine type given pricing and cost constraints